Lab 10 — Uitbating en goede gewoonten

Theorie — wat er na de `up` gebeurt: observeren, begrenzen, bewaken, verharden, verspreiden, opruimen, en vanzelf laten herstarten. Het lab dat van "bij mij draait het" "het draait, en we weten waarom" maakt.

Doelstellingen

- Een container in productie observeren: `logs`, `inspect`, `top`, `stats`, `events`.
- CPU- en geheugenlimieten opleggen, de OOM killer begrijpen en het verband met de JVM.
- Een `HEALTHCHECK` schrijven en weten wat Podman ermee doet (imageformaat, systemd-timers).
- De basisverhardingen toepassen: niet-root, alleen-lezen, capabilities, `no-new-privileges`.
- Een tagstrategie kiezen, naar een privéregistry pushen, opruimen zonder te breken, bij het opstarten starten met Quadlet.

---

1. Observeren

Een draaiende container levert vier informatiebronnen, elk met een eigen commando:

Vraag	Commando	Wat je erin leest
Wat zegt de applicatie?	`podman logs -t --since 10m --tail 50 api`	haar standaard- en foutuitvoer, met tijdstempel
Hoe is hij gestart?	`podman inspect api`	image, commando, variabelen, mounts, limieten, toestand, gezondheid
Wat doet hij nu?	`podman top api`, `podman stats --no-stream`	processen, **werkelijk** verbruikte CPU en geheugen
Wat is er gebeurd?	`podman events --since 1h --filter container=api`	aanmaak, start, stop, `health_status`, OOM, verwijdering

De logs zijn de hoofdbron, en de regel van lab 06 geldt: de applicatie schrijft naar **stdout/stderr**, de engine vangt op. Podman bergt ze standaard op in één bestand per container (driver `k8s-file`, pad zichtbaar in `inspect` onder `LogConfig.Path`); Docker gebruikt `json-file`. In beide gevallen vult een bestand dat onbeperkt groeit uiteindelijk de schijf: `--log-opt max-size=10m` (Docker) of `log_size_max` in `containers.conf` (Podman) begrenzen het. In het bedrijf leest een agent (Fluent Bit, Promtail, Filebeat) die bestanden en stuurt ze naar een centraal platform (Loki, Elasticsearch) — de container zelf weet daar niets van.

> **PODMAN**
> Zonder daemon is er geen centrale gebeurtenisstroom: `podman events` leest een journaal dat elk commando wegschrijft (`events_logger`: `journald` op een Ubuntu met systemd, anders `file`). En `podman stats` vereist **cgroup v2-delegatie**: op WSL met `systemd=true` is dat in orde; zonder systemd antwoordt Podman `Error: unable to get cgroup stats` of toont het absurde geaggregeerde waarden.

2. Begrenzen

Zonder limiet kan een container al het geheugen van de machine opeisen en de andere mee de dieperik in trekken. `--memory 256m` plafonneert het geheugen, `--cpus 0.5` beperkt tot een halve kern, `--pids-limit 200` verbiedt de forkbom. De drie zijn af te lezen in `inspect` (`HostConfig.Memory`, `NanoCpus`, `PidsLimit`) en worden toegepast via de cgroups.

→ LINUX

De **cgroups v2** ordenen de processen in een boom; elke knoop draagt controlebestanden: `memory.max` (plafond), `memory.current` (gebruik), `cpu.max` (quotum/periode), `pids.max`. De container is een knoop in die boom, en `cat /sys/fs/cgroup/memory.max` **vanuit de container** toont zijn eigen limiet. Wanneer `memory.current` `memory.max` bereikt en er niets meer vrij te maken valt, activeert de kernel de **OOM killer**: hij doodt het meest vraatzuchtige proces van de groep, met `SIGKILL`. De container eindigt met `137`, en `inspect` zegt `OOMKilled: true`.

→ JAVA

Sinds Java 10 leest de JVM de cgroups: ze ziet `256m`, niet de 32 GB van de machine, en dimensioneert haar heap daarop. Standaard neemt de heap **25 %** van het containergeheugen (de helft bij een kleine limiet, onder 512 MB); `-XX:MaxRAMPercentage=75` brengt dat op 75 %, en laat de rest aan de *metaspace*, de threads en de native code. Doorgegeven via `JAVA_TOOL_OPTIONS` (lab 08) raakt die instelling de image niet. Een JVM waarvan de heap de limiet overschrijdt, werpt geen `OutOfMemoryError`: ze wordt gedood door de kernel. De twee cijfers, containerlimiet en heappercentage, beslis je samen.

3. Bewaken: de `HEALTHCHECK`

Lab 09 definieerde de healthchecks in Compose. Je kunt ze ook in de image graveren: `HEALTHCHECK --interval=5s --timeout=3s --retries=3 CMD wget -qO- http://localhost:8080/actuator/health || exit 1`. Elke container die vanuit die image start, draagt dan een toestand `healthy` / `unhealthy` in `podman ps`, zonder dat wie uitrolt eraan hoeft te denken — en Compose of Kubernetes kunnen hem overschrijven.

PODMAN

Twee valkuilen eigen aan Podman. Eén: Buildah maakt standaard images in het **OCI**-formaat, dat geen veld `HEALTHCHECK` kent; de instructie wordt **genegeerd**, met een enkele, gemakkelijk te missen waarschuwing bij het bouwen (`HEALTHCHECK is not supported for OCI image format`), en `podman healthcheck run` antwoordt `has no defined healthcheck`. `podman build --format docker` bewaart ze. Twee: in rootless wordt de periodieke controle uitgevoerd door een **systemd-timer** per container; zonder systemd-sessie blijft de toestand `starting`. Docker heeft geen van beide valkuilen — maar deelt de rest: een healthcheck **herstart** nooit de container, hij verandert alleen zijn toestand. Een orchestrator (of een `--restart` gecombineerd met `--health-on-failure=kill`, eigen aan Podman) grijpt in.

4. Verharden

Een container in productie heeft vijf beveiligingsinstellingen die je nakijkt, en geen enkele kost code:

1. **Niet-root in de image**: `USER 1000:1000` (lab 05). In rootless zou die `root` al jouw gebruiker zijn; op een Docker als root is het het verschil tussen een ontsnapping uit de container en een gecompromitteerde machine.

2. **Alleen-lezen bestandssysteem**: `--read-only`, met `--tmpfs /tmp` voor wat moet kunnen schrijven. Een aanvaller kan geen binary neerzetten en de applicatie niet wijzigen.
3. **Minimale capabilities**: `--cap-drop ALL`, en dan `--cap-add` van wat echt ontbreekt (zelden iets voor een API).
4. **Geen escalatie**: `--security-opt no-new-privileges` verbiedt *setuid*-binaries om rechten te winnen.
5. **Limieten**: geheugen, CPU, PID's (sectie 2) — een bescherming tegen denial of service, intern of extern.

> **→ BEVEILIGING**
> De **capabilities** verdelen de macht van root in een veertigtal stukken: `CAP_CHOWN` (eigenaar wijzigen), `CAP_NET_BIND_SERVICE` (poorten < 1024), `CAP_SYS_ADMIN` (bijna alles)… Een container krijgt er standaard een tiental; `--cap-drop ALL` haalt ze allemaal weg, en `chown` antwoordt dan `Operation not permitted`, zelfs voor root. Dat is het beginsel van het **minste privilege** toegepast op de kernel: wat niet toegekend is, kan niet misbruikt worden.

Diezelfde opties vind je terug in Compose (`read_only`, `cap_drop`, `security_opt`) en in Kubernetes (`securityContext`). Ze vervangen de imagescan niet (Trivy, Grype), noch de updates van de basisimage: een CVE in `glibc` los je op door te herbouwen, niet door te verharden.

5. Verspreiden: tags, digests, registry

Een **tag** is een verplaatsbare wijzer; een **digest** is een onveranderlijke vingerafdruk (lab 02). De klassieke bedrijfsstrategie: de CI pusht `api:1.4.2` (semantische versie), `api:1.4` en `api:1` (glijdende wijzers), plus `api:git-a1b2c3d` (traceerbaar naar de commit); `latest` wordt nooit uitgerold; en de productie-uitrol verwijst naar de **digest** (`api@sha256:…`), de enige garantie dat wat draait, is wat getest werd. Een basistag (`postgres:16-alpine`) leg je op dezelfde manier vast vóór een productierelease.

De privéregistry is de draaischijf: de CI pusht ernaar, de servers halen eruit, de scanners lezen erin. `registry:2` is de minimale vorm; in het bedrijf vind je Harbor, de registry van GitLab, Nexus, Artifactory, of die van de clouds — allemaal spreken ze dezelfde API `/v2/`. Een registry dient ook als **cache** voor publieke images, om niet af te hangen van Docker Hub en zijn downloadlimieten.

> **PODMAN**
> Podman eist standaard **TLS**, zelfs voor `localhost`: `podman push localhost:5000/api:1.0` faalt met `http: server gave HTTP response to HTTPS client`. `--tls-verify=false` laat het toe voor een testregistry; voor echt declareer je de registry `insecure = true` in `registries.conf`, of beter, geef je hem een certificaat. Docker stelt dezelfde eis, opgeheven via `insecure-registries` in `daemon.json`.

6. Opruimen

Een werkstation of CI-agent stapelt op: tussenliggende images `<none>` (*dangling*), gestopte containers, verweesde volumes, buildcache. `podman system df` becijfert elke categorie en wat terug te winnen valt. Daarna:

Commando	Verwijdert	Risico
`podman image prune`	de dangling images	laag: niets met een naam
`podman image prune -a`	**alle** images zonder container	hoog: haalt `postgres`, `api:1.0`... weg, opnieuw te downloaden of te bouwen
`podman container prune`	alle gestopte containers	gemiddeld: een container die "voor later" gestopt is, verdwijnt
`podman volume prune`	alle volumes die geen enkele container gebruikt	**zeer hoog**: een database tussen twee `up`'s is "ongebruikt" (lab 06)
`podman system prune`	gestopte containers + ongebruikte netwerken + dangling images (+ `-a`, `--volumes`)	stapelt de vorige op

VALKUIL
`prune` redeneert in "gebruikt door een bestaande container", nooit in "kostbaar". Vóór elke verwijdering van een volume: `podman volume ls --filter dangling=true` lezen en **bij naam** verwijderen. Op een werkstation volstaan `podman image prune` en `podman container prune` bijna altijd; `--volumes` en `-a` bewaar je voor een machine die leeg mag worden.

7. Starten bij het opstarten: Quadlet

Lab 03 toonde het: zonder daemon overleeft `--restart` `wsl --shutdown` niet. Het antwoord van Podman is **Quadlet**: een bestand `~/.config/containers/systemd/api.container`, geschreven als een systemd-unit met een sectie `[Container]` (`Image=`, `PublishPort=`, `Environment=`, `Volume=`...), dat `systemctl --user daemon-reload` omzet in een dienst. `systemctl --user start api` start hem, `enable` activeert hem bij het opstarten, `journalctl --user -u api` toont zijn logs. Dezelfde bestanden bestaan voor netwerken, volumes, pods en Kubernetes-bestanden (`.kube`).

→ WINDOWS / WSL
Een `--user`-dienst start alleen als je gebruikerssessie bestaat. Om ze te laten bestaan zonder aanmelding: `loginctl enable-linger $USER`. En WSL start de distributie pas bij het openen van een terminal (of via een geplande Windows-taak): "bij het opstarten" betekent "bij de eerste `wsl` van de dag". Voor een echte server die altijd aanstaat: Quadlet op een native Linux of een Kubernetes-cluster.

8. In het bedrijf: de plaats in een CI/CD-keten

Alles wat je gezien hebt, past in een *pipeline*: (1) `podman build` van een image, **één keer**, met `--format docker` als een `HEALTHCHECK` verwacht wordt; (2) **scan** op kwetsbaarheden, die blokkeert bij kritieke CVE's; (3) `push` naar de privéregistry met de versie- en committags; (4) uitrol in acceptatie **via digest**, integratietests met Compose; (5) promotie van **dezelfde digest** naar productie, door de orchestrator. De limieten, de verharding en de healthcheck staan in de manifesten, niet met de hand getypt. En de dagelijkse uitbating — gecentraliseerde logs, metrieken, alarmen — steunt op wat de container blootstelt: `stdout`, `/actuator/health`, `/actuator/prometheus`.

Onthouden

- `logs`, `inspect`, `top` / `stats`, `events`: vier vragen, vier commando's; de logs gaan naar stdout en worden door een agent gecentraliseerd.
- `--memory` / `--cpus` / `--pids-limit` werken via de cgroups; geheugen overschrijden = OOM killer, code `137`, `OOMKilled: true`; de JVM dimensioneert haar heap op de limiet (`MaxRAMPercentage`).
- `HEALTHCHECK` in de image vereist `--format docker` met Podman, en systemd-timers in rootless; hij meldt, hij herstart niet.
- Verharden: `USER`, `--read-only` + `--tmpfs`, `--cap-drop ALL`, `no-new-privileges`, limieten — en de image scannen.
- Versie- en committags, nooit `latest` in productie, uitrollen via digest; privéregistry met TLS (`--tls-verify=false` alleen om te testen).
- `prune` kent "kostbaar" niet: eerst `system df`, volumes bij naam verwijderen.
- Quadlet vervangt `--restart` voor het starten bij het opstarten; op WSL: `enable-linger` en een gestarte `wsl`.

Woordenschat

logdriver: `k8s-file` (Podman) / `json-file` (Docker), één bestand per container. — **cgroups v2**: controleboom van de resources (`memory.max`, `cpu.max`, `pids.max`). — **OOM killer**: kernelmechanisme dat een proces doodt wanneer het geheugen op is. — **`MaxRAMPercentage`**: aandeel van het containergeheugen voor de Java-heap. — **`HEALTHCHECK`**: gezondheidscommando gegraveerd in de image (docker-formaat). — **capabilities**: opdeling van de rootprivileges. — **`no-new-privileges`**: verbiedt escalatie via setuid. — **glijdende tag / digest**: verplaatsbare wijzer / onveranderlijke vingerafdruk. — **privéregistry**: imagedepot van het bedrijf, ook cache van publieke images. — **dangling**: image zonder tag en zonder container. — **Quadlet**: bestanden `.container` / `.pod` / `.kube` omgezet in systemd-units. — **linger**: systemd-gebruikerssessie die zonder aanmelding blijft bestaan.